FECAP – Fundação Escola de Comércio Álvares Penteado

Análise e Desenvolvimento de Sistemas

K F K Λ

Programação Orientada a Objetos

Grupo KFKA Tech

Vitor de Moura Lima

João Henrique Percino Albuquerque

Gabriel Barbosa da Silva Cabral

1. Visão geral

O KFKA é um sistema de acompanhamento escolar para o Ensino Fundamental. O professor lança o acompanhamento bimestral de cada aluno, a coordenação revisa e publica, e o responsável acompanha os relatórios, confirma a leitura e pode enviar uma observação. O front-end é em React e o back-end em Node.js, com Express e MySQL.

No back-end, todo o acesso ao banco passa pelo arquivo **back_end/src/models/sistemaKFKA.js**, que concentra duas classes: uma genérica, usada por qualquer cadastro, e a classe principal, chamada pelo **server.js** em cada requisição.

Classes do arquivo sistemaKFKA.js

Classe	Linha	Responsabilidade
EntidadeBase	26	Operações que todo cadastro precisa: listar, buscar por id e remover
SistemaKFKA	47	Classe principal usada pelo **server.js**; monta um **EntidadeBase** para cada uma das 7 tabelas e concentra login, cadastros e o acompanhamento bimestral

No construtor de **SistemaKFKA**, turmas, disciplinas, professores, responsáveis, alunos, tags e bimestres ganham cada um o seu próprio objeto **EntidadeBase**. Isso evita reescrever o SQL de listar, buscar e remover sete vezes — um para cada tabela.

2. Abstração

> Abstração é esconder como uma operação é feita por trás de um nome simples. Quem usa a classe não precisa saber qual tabela está por trás, só o que o método faz.

EntidadeBase guarda o nome da tabela e implementa as três operações que qualquer cadastro precisa. Quem usa essa classe só chama **listarTodos()**, **buscarPorId()** ou **remover()**, sem precisar escrever o SQL de novo para cada tabela.

```
    return rows;
  }

  async buscarPorId(id) {
    const [rows] = await pool.query(`SELECT * FROM ${this.tabela} WHERE id = ?`, [id]);
    return rows[0] || null;
  }

  async remover(id) {
    const [res] = await pool.query(`DELETE FROM ${this.tabela} WHERE id = ?`, [id]);
    return res.affectedRows > 0;
  }
}
```

O método **listarEntidade()** segue a mesma ideia: ele recebe o nome da rota (**'turmas'**, **'alunos'**, etc.) e usa **this[rota]** para pegar o objeto certo, sem precisar de um **if** para cada tabela.

```
213
214   async listarEntidade(rota) {
215     if (!this[rota]) throw new Error('Entidade inválida.');
216     return await this[rota].listarTodos();
217   }
```

Em **atualizarEntidade()** acontece algo parecido: quando a mudança é só ativar ou desativar um registro, um trecho só resolve para as 7 tabelas, sem precisar saber qual delas é.

```
299
300     throw new Error('Rota de criação inválida.');
301   }
302
303   async atualizarEntidade(rota, id, dados) {
304     const atual = await this.buscarEntidadePorId(rota, id);
305
306     if (Object.keys(dados).length === 1 && dados.ativo !== undefined) {
307       await pool.query(`UPDATE ${rota} SET ativo = ? WHERE id = ?`, [Number(dados.ativo), id]);
308       return await this.buscarEntidadePorId(rota, id);
309     }
310
```

3. Encapsulamento

> Encapsulamento é proteger a lógica interna de um objeto, deixando que ela só seja acessada por métodos específicos, sem expor os detalhes pra quem está usando a classe.

gerarHashSenha() e **senhaConfere()** ficam fora de qualquer classe, declaradas no topo do arquivo, e nenhum outro arquivo do sistema chama elas diretamente. Só o método **autenticar()** usa essas funções, e é ele quem decide se precisa criar o login de um responsável no primeiro acesso.

```
87          } else {
88            const [ins] = await pool.query(
89              `INSERT INTO usuarios (nome, identificador, email, senha, perfil, ativo)
90               VALUES (?, ?, ?, ?, 'responsavel', 1)`,
91              [resp.nome, resp.cpf, resp.email, gerarHashSenha(SENHA_PADRAO)]
92            );
93            rows = [
94              {
95                id: ins.insertId,
96                nome: resp.nome,
97                identificador: resp.cpf,
98                email: resp.email,
99                perfil: 'responsavel',
100               ativo: 1,
101               senha: gerarHashSenha(SENHA_PADRAO)
```

No mesmo método também fica a verificação de senha (com comparação segura contra ataques de tempo, via **crypto.timingSafeEqual**) e a checagem de conta ativa. O **server.js** só chama **autenticar(identificador, senha, perfil)** e recebe o usuário ou um erro — não sabe nem precisa saber como a senha foi conferida.

```
122             [prof.ra]
123           );
124           rows = uRows;
125         }
126       }
127
128       if (!rows.length) {
129         throw new Error('Usuário ou matrícula não encontrados para este perfil.');
130       }
131
132       const usuario = rows[0];
133       if (!senhaConfere(senhaLimpa, usuario.senha)) {
134         throw new Error('Senha incorreta. Se for seu primeiro acesso, clique em "Criar / Definir Senha".');
135       }
136
137       if (!String(usuario.senha).startsWith('scrypt$')) {
138         await pool.query(`UPDATE usuarios SET senha = ? WHERE id = ?`, [gerarHashSenha(senhaLimpa), usuario.id]);
139       }
140
141       if (!usuario.ativo) {
142         throw new Error('Esta conta está inativa. Contate a coordenação.');
143       }
```

Esse mesmo padrão de busca por CPF, e-mail ou matrícula do aluno aparece de novo em **definirSenhaResponsavel()**, que resolve sozinho se precisa criar ou só atualizar o usuário correspondente.

```
    const [respRows] = await pool.query(
      `SELECT r.* FROM responsaveis r
       LEFT JOIN alunos a ON (a.responsavel = r.nome OR a.nome = r.aluno)
       WHERE r.cpf = ? OR r.email = ? OR a.matricula = ?
       LIMIT 1`,
      [idLimpo, idLimpo, idLimpo]
    );

    if (!respRows.length) {
      throw new Error(
        'Responsável ou matrícula não encontrados! Peça ao Administrador para cadastrar o responsável primeiro.'
      );
    }

    const resp = respRows[0];

    const [existentes] = await pool.query(
      `SELECT id FROM usuarios WHERE identificador = ? AND perfil = 'responsavel'`,
      [resp.cpf]
    );
```

4. Regras de negócio encapsuladas

> Encapsular também serve pra concentrar uma regra de negócio em um único lugar, de um jeito que nenhuma outra parte do sistema consiga ignorar ou pular essa regra.

A regra de que o professor só pode lançar notas com o bimestre aberto fica dentro de **_validarAcompanhamento()**, chamada tanto na criação quanto na atualização de um acompanhamento. Os valores padrão de turma, ano e professor ficam centralizados logo na assinatura de **criarAcompanhamento()**.

```
        throw new Error(`O período de digitação do ${bimestre} está fechado. Fale com a coordenação.`);
      }
    }
  }

  async criarAcompanhamento(dados) {
    const {
      aluno,
      disciplina,
      turma = '6º Ano B',
      bimestre,
      ano = '2026',
      professor = 'Julio Cesar',
      media,
      tags = [],
      descricao,
      status = 'Rascunho',
      usuario_acao = professor,
      perfil = 'professor'
    } = dados;
```

Depois de criar o registro, o método devolve o resultado chamando **buscarAcompanhamentoPorId()** — o mesmo método que **atualizarAcompanhamento()** usa logo depois pra carregar o registro atual antes de aplicar as mudanças.

```
525
526       return await this.buscarAcompanhamentoPorId(res.insertId);
527     }
528
529     async atualizarAcompanhamento(id, dados) {
530       const atual = await this.buscarAcompanhamentoPorId(id);
```

5. Um ponto que ainda pode evoluir: herança e polimorfismo

> Herança é quando uma classe reaproveita atributos e métodos de outra usando **extends**. Polimorfismo é quando o mesmo método se comporta de um jeito diferente em cada classe, porque a classe filha sobrescreve o método da mãe. No arquivo atual não existe nenhum dos dois — só **EntidadeBase** e **SistemaKFKA**, sem **extends** em lugar nenhum.

Isso fica claro no método **criarEntidade()**. Em vez de cada cadastro ter a sua própria classe herdando de **EntidadeBase** e sobrescrevendo um **inserir()**, o método resolve os sete casos com uma sequência de **if (rota === ...)**, um bloco de SQL pra cada tabela.

```
226   async criarEntidade(rota, dados) {
227     if (rota === 'turmas') {
228       const { ano = '2026', nome, turno, capacidade, disciplinas } = dados;
229       const [res] = await pool.query(
230         `INSERT INTO turmas (ano, nome, turno, capacidade, disciplinas) VALUES (?, ?, ?, ?, ?
231         [ano, nome, turno, Number(capacidade), disciplinas]
232       );
233       return await this.turmas.buscarPorId(res.insertId);
234     }
235
236     if (rota === 'disciplinas') {
237       const { nome, area } = dados;
238       const [res] = await pool.query(
239         `INSERT INTO disciplinas (nome, area) VALUES (?, ?)`,
240         [nome, area]
241       );
242       return await this.disciplinas.buscarPorId(res.insertId);
243     }
```

atualizarEntidade() tem o mesmo formato, só que com **else if**: cada bloco monta o **UPDATE** de uma tabela diferente. Com uma classe por cadastro, cada uma com o seu **atualizar()**, o **SistemaKFKA** chamaria só **repositorio.atualizar()** e deixaria cada classe cuidar do próprio SQL.

```
323     } else if (rota === 'disciplinas') {
324       const { nome = atual.nome, area = atual.area } = dados;
325       await pool.query(`UPDATE disciplinas SET nome = ?, area = ? WHERE id = ?`, [nome, area, id]);
326     } else if (rota === 'professores') {
327       const {
328         nome = atual.nome,
329         ra = atual.ra,
330         email = atual.email,
331         turma = atual.turma,
332         disciplina = atual.disciplina
333       } = dados;
334       const aulas = `${turma} • ${disciplina}`;
335       await pool.query(
336         `UPDATE professores SET nome = ?, ra = ?, email = ?, turma = ?, disciplina = ?, aulas = ? WHERE id = ?`,
337         [nome, ra, email, turma, disciplina, aulas, id]
338       );
339     } else if (rota === 'responsaveis') {
340       const {
341         nome = atual.nome,
342         email = atual.email,
343         telefone = atual.telefone,
344         cpf = atual.cpf,
345         parentesco = atual.parentesco,
346         aluno = atual.aluno,
347         senha
348       } = dados;
349       await pool.query(
350         `UPDATE responsaveis SET nome = ?, email = ?, telefone = ?, cpf = ?, parentesco = ?, aluno = ? WHERE id = ?`,
351         [nome, email, telefone, cpf, parentesco, aluno, id]
352       );
```

É o próximo passo natural do projeto: transformar **EntidadeBase** em classe mãe e criar uma filha por cadastro (**Turmas**, **Disciplinas**, **Professores**...), cada uma com seu próprio **inserir()** e **atualizar()**, acabando com os blocos repetidos das Figuras 9 e 10.

6. Outros conceitos utilizados

Conceito	Onde aparece
Classe e objeto	**module.exports = new SistemaKFKA()** cria o único objeto usado pelo servidor; o construtor dele cria um objeto **EntidadeBase** para cada tabela.
Composição	**SistemaKFKA** tem sete objetos **EntidadeBase** (um por cadastro) e delega a eles as operações de listar, buscar e remover.
Construtor	**EntidadeBase** recebe o nome da tabela no construtor; **SistemaKFKA** usa o seu construtor pra montar os sete repositórios.
Funções auxiliares fora das classes	**gerarHashSenha()** e **senhaConfere()** funcionam como utilitários de apoio ao módulo, usados só internamente.

Conclusão

Dá pra ver que o back-end do KFKA já usa Programação Orientada a Objetos pra reaproveitar código: a classe **EntidadeBase** evita reescrever o CRUD básico sete vezes, e métodos como **listarEntidade()** tratam qualquer cadastro de forma genérica. O encapsulamento protege a lógica de senha e as regras do acompanhamento bimestral, concentrando tudo em métodos específicos em vez de espalhar pelo sistema.

O que falta é herança e polimorfismo: hoje **criarEntidade()** e **atualizarEntidade()** resolvem os sete cadastros com **if**/**else if** repetidos, quando uma classe filha por cadastro resolveria isso de um jeito mais limpo. Fica registrado aqui como o próximo passo do projeto.

O código completo, as instruções de execução e o histórico de commits estão no **README.md** do repositório.